Learning to Place Guards by Reinforcement: A Geo-Free Neural Policy for the Vertex-Guard Art Gallery Problem

Domagoj Ševerdija Jurica Maltar Nathan Chappel
Domagoj Matijević

Abstract

Neural combinatorial optimization has shown that policies trained by reinforcement can construct strong solutions to NP-hard problems directly from raw instances; what such a policy actually learns about a problem's structure — as opposed to what its output decoder happens to express — remains much less clear. We study this distinction on the vertex-guard Art Gallery Problem, the NP-hard task of choosing polygon vertices from which to observe an entire region. A pointer-network policy is trained from a coverage-aware reward over its own rollouts under a constraint we call *geo-free* inference: at test time the model sees only the polygon's vertex coordinates, with no visibility computation and no geometric oracle. The policy places guards economically but leaves a tail of under-covered polygons that widens on instances far larger than any seen during training. To locate the cause of this tail — a limit of what the policy has *learned*, or only of what its decoder *expresses* — we freeze the trained encoder and read its embeddings with a small single-shot classifier, still geo-free at inference. The classifier closes most of the feasibility gap, in and out of distribution and at vertex counts reaching roughly seven times the training range, cutting the rate of under-covered polygons by about an order of magnitude at a controlled and explicitly reported cost in guard count. We read this as evidence that the reinforcement-trained representation already encodes much of the geometry a feasibility decision requires, and that the policy's residual failures reflect decoder calibration rather than missing knowledge. Beyond the Art Gallery Problem, freezing a learned encoder and probing it offers a concrete, reusable way to ask what a neural combinatorial solver has internalized.

1 Introduction

The Art Gallery Problem (AGP) asks for the minimum number of guards needed to observe every point of a simple polygon [10, 12], with applications in surveillance, sensor placement, and robotics [6, 3]. We focus on the *vertex-guard* variant, in which guards must be chosen from the polygon's vertices; the problem

remains NP-hard and is a discrete instance of geometric set cover with non-uniform, polygon-dependent visibility regions. The question this paper investigates is whether a neural policy can learn to place such guards from coordinates alone, trained from a coverage-aware reward signal over its own rollouts, with no visibility matrix and no geometric oracle at the moment it chooses guards. We are not asking whether a learned solver can match a classical greedy heuristic on guard count — it cannot, and we do not claim it does. We are asking whether the underlying combinatorial-geometric task is learnable when the model is denied any explicit geometric input at test time.

The reinforcement method we evaluate is an LSTM pointer-network policy trained with Preference Optimization (PO) [13] on Bradley–Terry preference pairs [4] drawn from the policy's own rollouts. The policy emits a vertex-index sequence with an EOS token; the reward couples coverage and guard usage. PO replaces REINFORCE advantages [17, 1] with binary preferences between pairs of rollouts, sidestepping REINFORCE's variance and the brittleness of supervised cloning of search labels. We refer to inference under this policy as *geo-free* — the network reads only vertex coordinates at the moment it places guards, never a visibility oracle (defined precisely in Section 2.3).

The policy works, to a degree. On a held-out in-distribution split its guard sets are competitive in cardinality with classical baselines, but the per-polygon coverage distribution has a tail: a non-trivial fraction of polygons end up below the feasibility line one would want a deployed solver to clear, and the failure rate grows on out-of-distribution polygons whose vertex counts exceed the training range. The reinforcement method, by itself, is not feasibility-guaranteed.

To check whether this residual reflects a true limit of the reinforcement-learned representation or only a calibration problem in the decoder, we train a small per-vertex classifier that reads only the frozen encoder embeddings of the policy and predicts a vertex-inclusion probability, controlled by a single scalar threshold. The classifier has no access to visibility information at inference, and it compresses the feasibility tail substantially both in and out of distribution. We read this as evidence that the reinforcement-trained encoder already carries enough geometric structure to support a feasibility decision, and that the policy's coverage tail was a decoder-calibration limit rather than a representation limit. The cost of recovering feasibility is a controlled rise in guard count, which we report explicitly.

Contributions. This paper makes four contributions.

- **C1 A representation–decoder decomposition.** Freezing the reinforcement-trained encoder and training a single-shot per-vertex probe over its embeddings isolates what the learner *represents* from what its decoder *expresses*. The pattern is methodological and not specific to AGP: it turns "does the model know X" into a measurable question about a frozen representation.
- **C2 Out-of-distribution generalization at scale.** On polygons whose vertex counts reach roughly seven times the training range, the probe raises

the fraction meeting the 0.95 feasibility gate from 85% (policy seed) to 99%, averaged over four probe seeds (Section 6.3).

C3 Geo-free inference as a protocol. Restricting test-time inputs to vertex coordinates isolates what a learner internalizes from what a visibility oracle could compute for it, making "no explicit geometric knowledge" a measurable property rather than a rhetorical one (Section 2.3).

C4 A structural finding on post-processing. Iterative add/remove/stop editors fail to improve a strong seed in any coverage-preserving regime, whereas the single-shot probe is empirically a fixed point under iteration (Section 6.4); the contrast is what makes the single-shot design the one that exposes the encoder's information.

We do not claim to beat classical greedy or local-search heuristics on guard count; we claim, and document, that vertex-guard placement is learnable to a measurable degree by a reinforcement method that never reads an explicit visibility object at inference.

The remainder of the paper is organized as follows. Section 2 fixes notation and the geo-free constraint. Section 3 reviews related work. Section 4 describes the reinforcement method and the representation probe. Section 5 describes the experimental setup. Section 6 reports the results. Section 7 reads the results against the question. Section 8 states limitations and Section 9 concludes.

2 Problem and Notation

2.1 Polygons and visibility

Let $P \subset \mathbb{R}^2$ denote a simple polygonal region (the closed set consisting of its interior and boundary), with n vertices at coordinates $x_1, \ldots, x_n \in \mathbb{R}^2$ and vertex index set $V(P) = \{1, \ldots, n\}$. A point p *sees* a point q when the closed segment between them stays inside the polygon, $\overline{pq} \subseteq P$. Guards are placed at vertices: for a vertex-guard set $S \subseteq V(P)$, the visibility region and area-normalized coverage are

$$\mathrm{Vis}(S) \;=\; \{\, q \in P : \exists\, i \in S,\ x_i \text{ sees } q \,\}, \qquad \mathrm{Cov}(S) \;=\; \frac{\mathrm{Area}(\mathrm{Vis}(S))}{\mathrm{Area}(P)} \in [0,1]. \tag{1}$$

2.2 The vertex-guard variant

The *vertex-guard Art Gallery Problem (AGP)* asks for the smallest vertex-guard set that covers the whole polygon. We define the *vertex-guard optimum* directly:

$$\mathrm{OPT}(P) \;=\; \min\bigl\{|S| : S \subseteq V(P),\ \mathrm{Cov}(S) = 1\bigr\}. \tag{2}$$

Restricting guards to $V(P)$ reduces placement to a finite discrete choice over n vertices — the natural action space for a pointer-network policy [16, 1, 8, 9]

— while retaining the full NP-hardness and non-uniform visibility structure of the problem. The public instance library provides *point-guard* solutions (guards anywhere in P); these are not the vertex-guard optimum and are not used as OPT here (see Section 5.1).

2.3 Geo-free inference

We will say that an inference procedure is *geo-free* if its inputs at test time are limited to the polygon's vertex coordinates and learned features derived from them — no polygon visibility matrix, no CGAL-computed visibility regions, no per-vertex area or visibility-fraction feature. Geo-free inference does not preclude using exact visibility during evaluation (to report coverage after the fact) or during training (to compute rewards, gate trajectories, or build supervised targets); it restricts what the model reads at the moment it places guards. A solver allowed to query a visibility oracle at every decision step — as classical local search and active-search decoding both do — can in principle simulate greedy or local search, and a measurement of what such a solver has learned would be conflated with a measurement of what its oracle can compute. The geo-free constraint isolates the first question from the second, which is what makes "no explicit geometric knowledge" a measurable property and not just a rhetorical one.

3 Related Work

This section places the present work in three lines of research: classical algorithms for the Art Gallery Problem, neural combinatorial optimization with reinforcement learning, and supervised post-processing of learned solvers.

The Art Gallery Problem. Lee and Lin [10] established NP-hardness for several variants of the AGP; O'Rourke [12] surveys the foundational geometric and combinatorial results. The vertex-guard variant has a long history of approximation algorithms and exact integer-programming approaches; we treat published instance libraries [5] as the source of polygons in our experiments. Classical greedy heuristics [7] place guards to maximize marginal coverage and are reliable on small to mid-sized polygons; local search refines candidate guard sets by swapping or removing vertices — consulting exact visibility at every move — and is the standard high-quality baseline. We use these classical methods only as evaluation reference points and as a source of supervised labels for the representation probe in Section 4.2; they are not in competition with the reinforcement method on guard count.

Reinforcement learning for combinatorial optimization. Pointer networks [16] introduced the architecture of attending over input tokens to produce variable-length output index sequences, and were extended to combinatorial optimization with reinforcement learning by Bello et al. [1] using REIN-

FORCE [17] with a learned baseline. Subsequent work introduced attention-only architectures for routing problems [8] and policy optimization variants such as POMO [9] that exploit problem symmetries. We adopt the more recent preference-optimization (PO) recipe of Pan et al. [13], which trains the policy with a Bradley–Terry [4] ranking loss over pairs of solutions sampled from the policy itself, replacing the variance-heavy REINFORCE objective. We treat PO as a reinforcement-style procedure: the loss is computed from policy rollouts on the environment (the polygon), the gradient direction is determined by the reward (coverage versus guard usage), and no supervised labels for the action sequence are used. A broader methodological survey is provided by Bengio et al. [2].

Supervised post-processing of learned solvers. An orthogonal line of work post-processes a base solver's output with a second model trained by supervised or imitation learning. For combinatorial problems with hard feasibility constraints (such as covering), iterative supervised editors must contend with compounding errors and a train–inference distribution gap; DAgger [14] and its variants partially address the latter. We use a single-shot supervised post-processor (the `SetPredictor`) not as a competing solver but as a *probe* of the RL-trained encoder's representation. The framing is methodological: if a small classifier reading frozen RL embeddings can recover feasibility, the RL encoder must already carry the relevant geometric information. Directly cloning search solutions with a recurrent decoder, by contrast, fails on this problem for a structural reason — the teacher-forcing cascade we return to in Section 6.4 — which is why we read the frozen encoder with a single-shot, recurrence-free probe rather than a sequential supervised decoder.

4 Method

We describe the reinforcement method (Section 4.1) and the single-shot representation probe (Section 4.2); the empirical diagnostic that most motivates the single-shot design — why iterative editors fail to improve the seed — is deferred to the results (Section 6.4).

4.1 The reinforcement method: PO/BT pointer policy

Architecture. The policy is a compact LSTM-encoder pointer network (∼360k parameters) with an attention-based decoder. A polygon P with vertex sequence $V(P) = (x_1, \ldots, x_n)$ is consumed coordinate-wise (no visibility input). The encoder produces per-vertex embeddings $\{h_i\}_{i=1}^{n}$, $h_i \in \mathbb{R}^{128}$. The decoder attends over the encoder at each step, emits a vertex index or an EOS token, and stops at EOS or when the maximum length is reached. The decoder outputs a sequence $\pi = (\pi_1, \ldots, \pi_T)$; the resulting guard set is $S(\pi) = \{\pi_t : \pi_t \neq \text{EOS}\}$, and all metrics (coverage, $|S|/n$, $|S|/\text{OPT}$) are computed on $S(\pi)$. The encoder acquires its geometric representation only through the reward gradient

back-propagated from the decoder's rollouts: outside the policy that trained it there is no encoder to read, which is why we probe the representation *through* a frozen encoder rather than train a classifier on an encoder in isolation. The same decoder's greedy output is also the seed that the representation probe (Section 4.2) later refines.

Why an autoregressive pointer. Vertex-guard placement is a coverage (set-cover) problem: the value of a guard depends on what the already-chosen guards leave uncovered. Conditioning each pick on the partial solution factorizes the guard-set distribution $p(S \mid P) = \prod_t p(\pi_t \mid \pi_{<t}, P)$ into conditionals that mirror the greedy marginal-coverage rule known to be near-optimal for submodular cover, and it lets the policy decide cardinality endogenously through the EOS token rather than through a global threshold. A model that scored each vertex independently could not represent this conditional structure — which is precisely why the single-shot probe of Section 4.2 consumes the policy's seed as an input and serves as a representation *probe* rather than a standalone solver.

Reward and rollouts. For each polygon P we sample R rollouts from the policy and score each rollout's guard set $S(\pi)$ by a scalar utility that gates coverage at τ and then prefers smaller guard sets,

$$u(\pi \mid P) \;=\; \min\big(\mathrm{Cov}(S(\pi)),\, \tau\big) \;-\; \lambda\, \frac{|S(\pi)|}{n} \;-\; \rho\, \max\big(0,\, \tau - \mathrm{Cov}(S(\pi))\big), \quad (3)$$

where $\tau = 0.99$ is the coverage gate, λ weights guard usage, and ρ penalizes coverage below the gate. Capping the coverage term at τ removes any incentive to over-cover, so once a rollout is feasible the utility is driven purely by cardinality. Coverage during training is computed with a discretized-visibility sampler ($M = 500$ interior points per polygon) as a cheap approximation; CGAL exact visibility is reserved for evaluation.

Preference-optimization loss. For each pair (π^+, π^-) of rollouts with $u(\pi^+) > u(\pi^-)$ above a coverage gate, we apply the Bradley–Terry log-likelihood

$$\mathcal{L}_{\mathrm{BT}}(\theta) \;=\; -\mathbb{E}_{(P,\pi^+,\pi^-)}\left[\, \log \sigma\big(\alpha\,[\log p_\theta(\pi^+ \mid P) - \log p_\theta(\pi^- \mid P)]\big)\,\right], \quad (4)$$

where $\alpha > 0$ is a temperature scaling the preference margin and $\log p_\theta(\pi \mid P)$ is the *mean* per-token log-probability of the sequence — length-normalized so the preference is not biased by guard-set size, since AGP solutions have variable length (cf. Pan et al. [13]). The gradient direction is driven purely by which of the two rollouts achieved a higher reward; no supervised target sequence is used.

We use PO rather than REINFORCE deliberately, and Table 1 is the evidence. Pan et al. [13] report that binary preferences over pairs of rollouts retain a useful gradient direction even when rewards saturate — the regime a value-baseline REINFORCE [1] struggles with on dense-reward problems — and on

Training objective	Mean cov	Mean $\lvert S\rvert$/OPT
REINFORCE, learned-critic baseline	0.904	3.49
REINFORCE, greedy rollout baseline	1.000	7.00
PO / Bradley–Terry (ours)	**0.978**	**1.30**

Table 1: Why preference optimization. REINFORCE policies trained on the same polygon distribution cover well but cannot control guard count; PO/BT reaches comparable coverage at near-optimal cardinality. Coverage is mean geometric (CGAL) polygon coverage; $|S|$/OPT is the approximation ratio. The REINFORCE rows are 30-epoch runs evaluated under an earlier greedy-decode protocol on `dev`, reported as evidence for the design choice rather than as tuned competitors — the $|S|$/OPT gap exceeds any plausible protocol difference.

AGP we observe exactly this failure: a value-critic REINFORCE policy over-guards by roughly 3.5× the optimum and a greedy-baseline variant by about 7×, while PO/BT reaches comparable coverage at 1.3×. The cause is structural: under the coverage-gated reward of Eq. (3), once a rollout saturates coverage the policy-gradient signal on the guard-cost term vanishes, whereas Bradley–Terry preferences between rollouts stay informative even when both saturate, preserving a usable gradient on the cardinality axis. At the end of training we are left with a single deterministic policy that maps a polygon's coordinates to a guard sequence in one forward pass; this is the artifact whose properties the rest of the paper investigates, and from this point on its encoder is frozen and treated as a learned feature extractor. Figure 1 reports the held-out training dynamics across four reconstructed checkpoints and shows that coverage and cost move jointly over training rather than the optimization collapsing into a degenerate single-objective regime.

4.2 Probing the representation: a single-shot per-vertex classifier

The policy is strong on cardinality but leaves a feasibility tail (Section 6.1), and a learned *iterative* editor over its seed does not close that tail in any coverage-preserving regime (Section 6.4): error accumulation, stop-time miscalibration, and train/inference drift make a rollout-based post-processor unreliable here. We therefore probe what the policy has internalized with a *single-shot, rollout-free* classifier — conditioned only on the encoder's output and never on a visibility object at inference — that recovers the feasibility decision the decoder failed to make. We call it the `SetPredictor` and present it primarily as a representation diagnostic; the threshold sweep in Section 6 makes the diagnostic concrete, but the operating-curve framing is a side-effect of the measurement, not a deployment recipe.

For polygon P with vertex sequence $V(P) = (x_1, \ldots, x_n)$, the input feature for vertex i is

$$z_i \;=\; [\, h_i \,;\, x_i \,;\, \mathbf{1}\{i \in \pi_{\text{seed}}\} \,] \;\in\; \mathbb{R}^{131}, \tag{5}$$

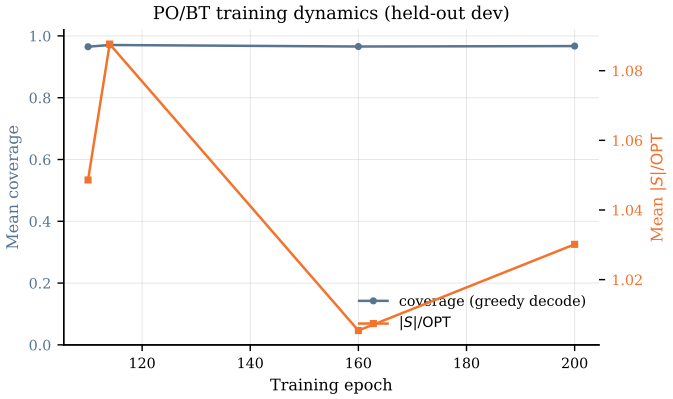

Figure 1: PO/BT training dynamics on the held-out `dev` split, reconstructed post-hoc from four saved checkpoints (see Section 8). Mean per-polygon coverage (left axis) and the approximation ratio $|S|/\mathrm{OPT}$ (right axis) of the greedy-decoded policy. The trace covers the late phase of training; the coverage–cost trade-off improves jointly rather than collapsing into a single-objective regime.

that is, the *frozen* 128-dimensional encoder embedding h_i from the PO/BT policy, the 2D coordinates x_i, and a single binary indicator for whether vertex i is selected by the policy's greedy-decoded seed π_{seed}. No visibility matrix, no CGAL output, and no area feature enters the classifier at inference time. The geo-free constraint is preserved.

The architecture is a small Transformer encoder over the n vertex tokens with a sigmoid output head per token ($\sim$460k parameters; layer, head, and width settings in Section 5.2). Writing v_i for the attention output at vertex i, we form a per-vertex representation by concatenating it with two pooled context vectors — mean-pooled over the seed set (S_{pool}) and over all valid (non-padded) vertices (G_{pool}) — and feed the concatenation to a 2-layer MLP head:

$$\hat{p}_i \;=\; \sigma\big(\mathrm{MLP}([\,v_i \,\|\, S_{\text{pool}} \,\|\, G_{\text{pool}}\,])_i\big), \qquad i = 1, \ldots, n. \tag{6}$$

The output $\hat{p}_i \in [0,1]$ is interpreted as the probability that vertex i should be in the final guard set. Thresholding at $t \in (0,1)$ yields the guard set

$$S(t) \;=\; \{\, i \in \{1,\ldots,n\} : \hat{p}_i \geq t \,\}. \tag{7}$$

Objective. The probe is trained by supervised learning to predict membership in an LS-improved target set $\pi^{\star}_{\text{LS}}(P)$ — the policy's seed refined by local search, constructed as described in Section 5.2 — so it is not itself a reinforcement learner. With $y_i = \mathbf{1}\{i \in \pi^{\star}_{\text{LS}}(P)\}$ and a positive-class weight w_+ that balances the empirical guard/non-guard ratio, the per-vertex weighted binary

cross-entropy is

$$\mathcal{L}(\theta) \;=\; -\frac{1}{|\mathcal{B}|}\sum_{P\in\mathcal{B}}\frac{1}{|V(P)|}\sum_{i=1}^{|V(P)|}\big[\,w_{+}y_i\log\hat{p}_i+(1-y_i)\log(1-\hat{p}_i)\,\big]. \qquad (8)$$

The logic is direct: if a classifier reading only the frozen embeddings h_i (with x_i and the seed indicator) can close the feasibility tail, the encoder must already carry the geometry that decision needs.

Inference and the threshold knob. At inference we run the policy encoder once on the coordinates, greedy-decode the seed π_{seed}, form the features z_i of Eq. (5), run one `SetPredictor` forward pass to obtain $\{\hat{p}_i\}_{i=1}^{n}$, and threshold to return $S(t)$ (Eq. (7)). No CGAL call, visibility matrix, or oracle appears on this path. The threshold t is the single knob trading coverage for guard count, swept over $t \in \{0.20, 0.25, 0.30\}$ in Section 6.2. Re-running the probe with its own output as the updated seed indicator changes nothing — it is empirically a fixed point after the first pass (Section 6.4) — so we use a single pass throughout.

5 Experiments

5.1 Dataset and partition

We draw polygons from the publicly available AGP instance library of Couto, de Rezende, and de Souza [5], which contains four families (random simple, random orthogonal, random von Koch, complete von Koch) over a wide range of vertex counts n. The library distributes *point-guard* optimal solutions — guards may be placed at any point of the polygon, not only at vertices — which are not the optimum for the vertex-guard variant we study. We therefore compute vertex-guard optima OPT ourselves, by solving the vertex-guard set-cover integer program for each polygon, and use these computed OPT values throughout the paper. The splits we work with are listed in Table 2.[1]

5.2 Training setup

The PO/BT policy is trained once and frozen — the existing `lstm_bt` checkpoint, with all PO coverage rewards computed from disc-vis sampling ($M = 500$) — and a single seed (1234). The `SetPredictor` probe is trained on `train` only. *Targets:* for each training polygon we obtain $\pi^{\star}_{\text{LS}}(P)$ by running local search on the policy's seed under a coverage gate $\tau = 0.99$ against a disc-vis reference; CGAL exact visibility is used only at evaluation. *Sizes:* the LSTM pointer policy has ∼364k parameters with 128-dimensional encoder embeddings; the `SetPredictor` is a 3-layer, 8-head Transformer encoder (hidden width 128, FFN

[1] The 70/30 `dev` partition uses a seeded random shuffle (seed 1234) rather than an alphabetical-name split: the latter had concentrated `fat-*` polygons in `dev_tune` and `randsimple-*` in `dev_test`, producing misleading optimism that the random shuffle removes.

Split	# polygons	n range	Use
train	9,791	8–150	Pointer + SetPredictor training
dev_tune	857	8–150	SetPredictor checkpoint selection
dev_test	367	8–150	Held-out evaluation (in-distribution)
test	2,107	12–1,000	Out-of-distribution evaluation
large	285	600–2,250	Extreme-OOD evaluation

Table 2: Dataset partition. The `dev` split is partitioned 70/30 into `dev_tune` (checkpoint selection) and `dev_test` (held-out evaluation) by seeded random shuffle (seed 1234). The `test` split contains polygons whose vertex count exceeds the training range and is used as the out-of-distribution evaluation. The `large` split is used for an extreme-OOD evaluation (Table 7).

expansion factor 4) with a 2-layer MLP head, 464,001 parameters in total. *Optimization:* weighted BCE (Eq. (8)) with positive-class weight $w_+ \approx 5.66$ (the empirical guard/non-guard ratio on `train`), 60 epochs, batch size 32, learning rate 3×10^{-4}, AdamW [11]. The headline probe is four independent runs with seeds $\{1234, 11, 22, 33\}$ (reported as mean $\pm$ std); the coordinate-only ablation and all figures use seed 1234. Checkpoints are selected on `dev_tune`; `dev_test`, `test`, and `large` are touched only after training and threshold selection are complete.

5.3 Evaluation protocol and metrics

For each polygon we greedy-decode the policy's EOS-truncated seed and evaluate it with CGAL exact polygon visibility [15]; separately, we run one `SetPredictor` pass, threshold at t to obtain $S(t)$, and evaluate $S(t)$ with CGAL. Let N be the partition size. We report mean coverage $\frac{1}{N}\sum \mathrm{Cov}(S)$, the normalized guard count $|S|/n$, and the approximation ratio $|S|/\mathrm{OPT}$ against the vertex-guard optimum. Because the mean hides the tail, we also report the per-polygon counts $\#\{\mathrm{Cov}(S) \geq c\}$ for $c \in \{0.95, 0.99, 0.999, 1\}$ and the worst case $\min_P \mathrm{Cov}(S)$; we call $\mathrm{Cov}(S) \geq 0.95$ the *feasibility* threshold, report the feasibility rate $\#\{\mathrm{Cov}(S) \geq 0.95\}/N$ with Wilson 95% confidence intervals, and define the *recovery rate* of a post-processor as the fraction of its edited solutions that match or beat the local-search reference on both coverage and guard count. The reporting threshold c is distinct from the training-time gate $\tau = 0.99$ of Eq. (3). As noted in Section 5.1, vertex-guard optima OPT are computed by us via integer programming (the public library distributes only point-guard solutions); OPT is available for `train`, `dev`, and `test`, but for 79 of the `large` polygons (all with $n \geq 800$) the ILP did not terminate within budget, so on `large` we report coverage and the normalized guard count $|S|/n$ but not the approximation ratio.

Method	Mean cov	#{Cov ≥ .95}/N	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert$/OPT
Classical baselines				
Greedy	0.9994	367/367	0.1523	1.0086
Local search	0.9948	367/367	0.1367	0.8840[†]
Learned policy / probe				
Pretrained pointer (seed)	0.9689	296/367 (0.763,0.844)	0.1664	1.0878
SetPredictor full ($t = 0.20$)	**0.9984** ± 0.0011	366.2 ± 1.3/367	0.3890 ± 0.0606	2.5593 ± 0.4043
SetPredictor, no-encoder ($t = 0.20$)	0.9952	367/367	0.3978	2.5783

Table 3: Held-out evaluation on `dev_test` (367 polygons). Classical greedy and local search are the standard non-learned anchors (greedy guard set from [12]-style farthest-coverage; LS is the trajectory used to generate `SetPredictor` targets). The *seed* row is the PO/BT-trained policy decoded greedily. ***`SetPredictor`*** *full* is the headline representation probe; numbers are mean ± std across four probe seeds $\{1234, 11, 22, 33\}$ at fixed threshold $t = 0.20$. ***`SetPredictor`****, no-encoder* has the same architecture and parameter count with the frozen encoder masked to zeros (single-seed contrast, Section 6.2). Wilson 95% CI is shown for the policy seed only, where the feasibility proportion is informative; rows that reach 367/367 omit the degenerate CI.
[†]Local search reports $\lvert S\rvert$/OPT < 1 because the OPT reference is a time-limited B&B lower bound, not a verified optimum (Section 5); $\lvert S\rvert/n$ is the cardinality metric without this caveat.

6 Results

We organize the results around the hypothesis. Section 6.1 reports what the policy achieves on its own; Section 6.2 reads the encoder to identify what closes the residual feasibility tail; Section 6.3 is the main out-of-distribution measurement; and Section 6.4 contrasts the failed iterative editors with the probe's fixed-point property.

6.1 The policy places guards

The geo-free policy on its own (Table 3, *seed* row) places guards at near-classical cardinality — its mean $\lvert S\rvert/n$ sits between greedy and local search, at $\lvert S\rvert$/OPT ≈ 1.09 — and clears the feasibility line on the large majority of `dev_test` polygons. Read against the introduction's question, this confirms that vertex-guard placement is learnable, to a degree, by a method that never reads an explicit visibility object at inference. The honest scope is the residual: 71 of the 367 polygons fall below the feasibility line under the policy alone (the distribution is Section 6.2), so the method gives no feasibility guarantee on its own. Figure 2 shows the seed, the seed plus probe at $t = 0.20$, and the optimum side by side for an in-distribution and an out-of-distribution polygon.

6.2 Reading the encoder closes the tail

The policy's mean coverage hides a bimodal distribution (Table 4): it reaches full coverage on only a handful of polygons and trails a long left tail down to

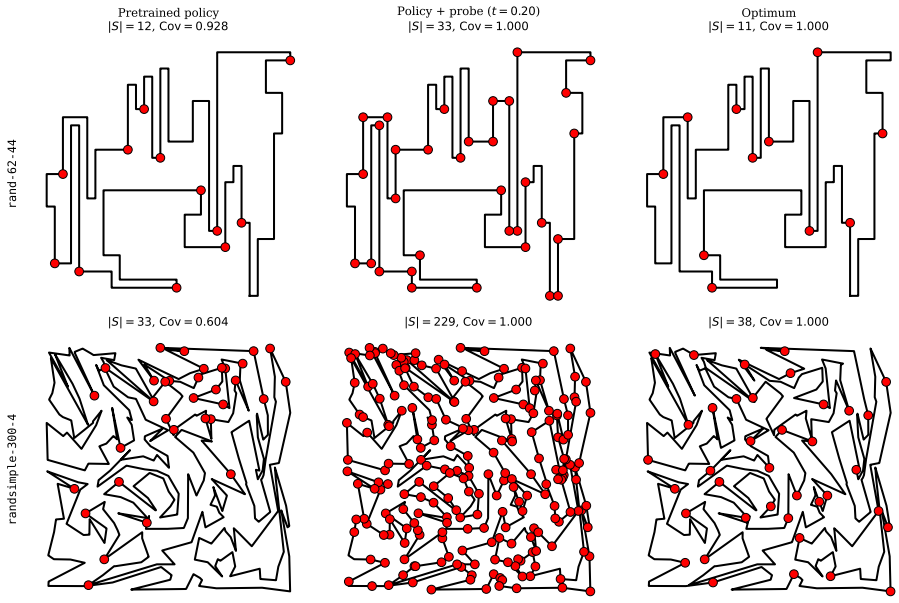

Figure 2: Worked examples: two polygons drawn with the policy's seed (left column), the policy plus the `SetPredictor` probe at $t = 0.20$ (middle column), and the optimum (right column). Vertices selected as guards are marked. Row 1 is an in-distribution polygon where the policy already covers most of the region. Row 2 is an out-of-distribution polygon where the policy's seed leaves a substantial uncovered region and the probe restores feasibility by adding vertices the policy missed.

a worst polygon at 0.830, while the `SetPredictor` at $t = 0.20$ eliminates that tail and lifts the bulk above 0.99.

Is closing that tail the encoder's doing, or could coordinates alone suffice? We isolate the encoder's contribution by retraining the probe with the frozen embedding h_i masked to zero on every forward pass — architecture, parameter count, optimizer, and training schedule identical (Section 5.2). On the headline mean the two probes are nearly indistinguishable (Table 3), but the mean is the wrong statistic: it is dominated by easy polygons both already cover, and the encoder's contribution concentrates in the tail and out of distribution. Under a stricter $\mathrm{Cov}(S) \geq 0.99$ gate on `dev_test` the full probe leaves roughly a fifth as many polygons as the no-encoder ablation ($\approx$11 versus 57); on the OOD `test` split the gap is qualitative, the full probe leaving 17.5 ± 12.5 polygons against the no-encoder probe's 81 (Table 6), the latter inheriting the policy seed's near-zero worst case. This is the direct measurement behind C1 — the frozen encoder carries geometric structure that raw coordinates do not reproduce, even with the decoder's full parameter budget.

Method	#{Cov = 1}	#{Cov ≥ .999}	#{Cov ≥ .99}	#{Cov ≥ .95}	min Cov
Pretrained pointer (seed)	3	15	99	296/367	0.830
SetPredictor $t = 0.20$	243	315	363	367/367	0.977
SetPredictor $t = 0.25$	195	283	361	367/367	0.962
SetPredictor $t = 0.30$	152	221	355	367/367	0.952

Table 4: Per-polygon coverage distribution on `dev_test` (367 polygons). Counts show the number of polygons reaching each coverage threshold; *min* Cov(S) is the worst polygon. The pretrained policy has a long left tail; the `SetPredictor` moves the entire distribution to the right. `SetPredictor` rows show the displayed seed (1234); Table 3 reports the four-seed mean±std on the same split at $t = 0.20$.

Threshold t	Mean Cov	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert$/OPT	#{Cov ≥ .95}/N
0.20	0.9993	0.4376	2.9007	367/367
0.25	0.9988	0.3802	2.5091	367/367
0.30	0.9979	0.3352	2.2030	367/367

Table 5: Cost of feasibility on `dev_test` (367 polygons): as the threshold t falls, coverage rises and so does the guard count. Shown for the displayed seed (1234); the four-seed mean $\lvert S\rvert$/OPT is 2.02 ± 0.21 at $t = 0.30$ and 2.56 ± 0.40 at $t = 0.20$.

Closing the tail has a price, with the threshold t as the knob: as t falls from 0.30 to 0.20 the approximation ratio climbs (Table 5) from roughly 2.0 to 2.6× optimum on the four-seed mean, against ≈ 1.1 for the policy alone. Figure 3 gives the full picture in and out of distribution: the coverage CDF sits to the right of the policy at every threshold, while the $\lvert S\rvert/n$ and $\lvert S\rvert$/OPT box plots show the matching cost. We report the trade-off rather than fix one operating point.

6.3 Out-of-distribution generalization

The strongest test of the representation is the OOD `test` split, whose polygons reach $n = 1000$ — about seven times the largest training polygon. Table 6 reports it: the policy alone drops to mean coverage 0.957 with 310/2107 polygons below the feasibility line (85% feasible), while the probe at $t = 0.20$ (four-seed mean) cuts that to 17.5 ± 12.5 (99% feasible) — roughly an order-of-magnitude reduction in failures — at mean coverage 0.995. This is the evidence behind C2: a small classifier over the frozen embeddings recovers feasibility on polygons far larger than any the encoder saw in training. The reduction is robust across seeds; what varies is where each seed's failures concentrate along the size axis, so at-scale generalization is partially seed-dependent, with mean coverage staying above 0.97 across the full n range. Figure 4 disaggregates coverage by polygon size, and the bottom row of Figure 3 shows the matching distributions.

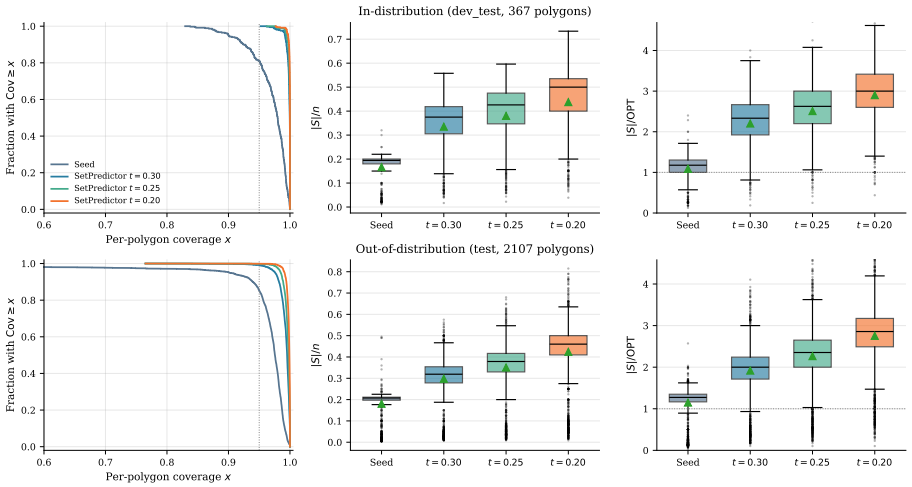

Figure 3: Distributional view of method performance, in-distribution `dev_test` (top, 367 polygons) and out-of-distribution `test` (bottom, 2107 polygons). *Left:* per-polygon coverage as a complementary stair-step CDF (fraction of polygons with coverage at least x); the dashed line marks the 0.95 feasibility gate. *Center, right:* box plots of the normalized guard count $|S|/n$ and the approximation ratio $|S|/\mathrm{OPT}$ for the policy seed and the three probe thresholds. The probe buys coverage at a controlled, visible cost in guard count. Boxes show the displayed seed (1234); Table 3 reports the four-seed mean±std at $t = 0.20$.

Extreme out-of-distribution. Pushing the same probe well past the `test` range onto the `large` split (n from 600 to 2250, up to about fifteen times the largest training polygon) sharpens the picture, and the headline is the tail rather than the mean (Table 7). The pretrained seed leaves its worst polygon at coverage 0.038 with 75/285 below the feasibility line, and the no-encoder ablation cannot move that worst polygon at all — its min Cov stays pinned at 0.038 at every threshold, so it only nudges the mean. The full probe, reading the frozen embeddings, lifts the worst polygon for every seed: across the four seeds the worst-polygon coverage at $t = 0.20$ averages 0.61 ± 0.29, and on the displayed seed reaches 0.951 with nothing left below feasibility. This is where the encoder signal is least ambiguous — the means alone understate it, because the no-encoder probe matches the full probe to within a few points on mean coverage (0.900 versus 0.963) while failing completely on the tail. The seed dependence is genuine: the four-seed feasibility count is 253.5 ± 19.9 of 285, with the per-seed count below feasibility ranging from 0 (the displayed seed) to 55 across the four seeds, so we read this as a strong-but-variable result rather than a guarantee. The cost is reported honestly — the probe spends about twice the guards of the local-search reference it imitates ($|S|/n \approx 0.30$ versus 0.15) and in return comfortably exceeds that reference's own feasibility rate on this

Method	Mean cov	#{Cov ≥ .95}/N	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert$/OPT
Pretrained pointer (seed)	0.9568	1797/2107 (0.837,0.867)	0.1807	1.1494
SetPredictor full ($t = 0.20$)	**0.9946** ± 0.0020	2089.5 ± 12.5/2107	0.3617 ± 0.0447	2.3367 ± 0.2948
SetPredictor, no-encoder ($t = 0.20$)	0.9765	2026/2107 (0.952,0.969)	0.2826	1.8096

Table 6: Out-of-distribution evaluation on `test` (2107 polygons, n up to 1000). Rows as in Table 3: the policy seed, the full probe (four-seed mean ± std at $t = 0.20$), and the no-encoder ablation. The full probe cuts sub-feasibility polygons from the policy's 310 to 17.5 ± 12.5; the no-encoder ablation leaves 81.

Method	Mean Cov	min Cov	#{Cov ≥ .95}/N	Mean $\lvert S\rvert/n$
Pretrained pointer (seed)	0.8678	0.038	210/285	0.1849
SetPredictor full ($t = 0.20$)	**0.9630** ± 0.0209	0.612 ± 0.286	253.5 ± 19.9/285	0.2951 ± 0.0579
SetPredictor, no-encoder ($t = 0.20$)	0.8997	0.038	231/285	0.2344

Table 7: Extreme out-of-distribution evaluation on the `large` split (285 polygons, n from 600 to 2250), coverage scored by exact CGAL at $t = 0.20$. The approximation ratio is omitted because the ILP optimum is unavailable for 79 of these polygons (all with $n \geq 800$; Section 5.3); the reported columns are over all 285. The full-probe row is the four-seed mean ± std, and its min Cov is the mean ± std of the four per-seed worst-polygon coverages; the no-encoder ablation is a single run. The story is the worst case: the no-encoder min Cov is pinned at the seed's 0.038, while the frozen-encoder probe lifts it.

split (61%), recovering coverage on instances far larger than any the encoder saw in training.

6.4 Iteration: editors drift, the probe is a fixed point

Before settling on the single-shot probe we tried to improve the seed with a learned *iterative* editor applying ADD/REMOVE/STOP edits. Three architectures of increasing capacity — including one with topology features and an auxiliary visibility-prediction loss, and one with DAgger refresh [14] — failed to improve over the seed in any coverage-preserving regime: the best variant cut $|S|$ by about 24% but lost coverage and reached only a 0.27 recovery rate against the local-search reference, well below replacement quality. Three structural reasons recur: errors accumulate because the editor sees clean trajectories in training but its own predictions at inference; a single $\{\text{ADD}, \text{REMOVE}, \text{STOP}\} \times \{1, \ldots, n\}$ head couples selection with termination, miscalibrating STOP; and the train/inference state distributions diverge in ways DAgger refresh did not close. The same teacher-forcing cascade also defeats a *supervised* pointer network trained to clone local-search sequences directly: forced onto sequences it would not itself generate, the recurrent decoder drifts into hidden states unseen at training and collapses (mean coverage ≈ 0.50 at roughly $3\times$ the optimum). The single-shot probe has none of these — no rollout, no stop token, trained and used in the same regime.

Re-running the probe with its own output as the updated seed indicator does

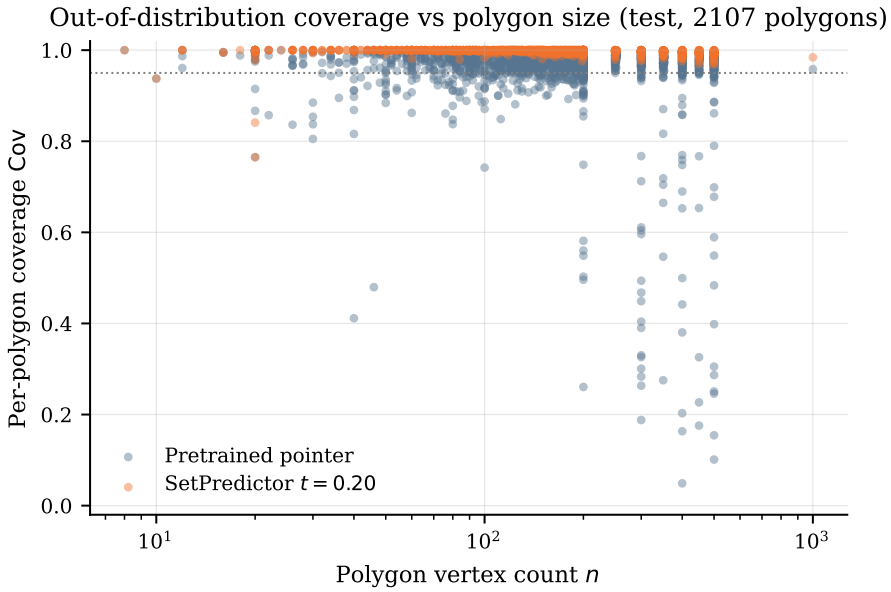

Figure 4: Per-polygon coverage as a function of vertex count n on the OOD `test` split ($n \in [12, 1000]$). The pretrained policy's coverage degrades systematically as n grows; the `SetPredictor` probe at $t = 0.20$ stays near the 0.95 feasibility line across the entire size range. Shown for the displayed seed (1234); across the four probe seeds the worst-polygon coverage at $t = 0.20$ ranges from 0.576 to 0.884 (Section 6.3).

not change the prediction: across $K \in \{1, 2, 3, 5\}$ the changes in mean coverage, $|S|/n$, and $|S|/\mathrm{OPT}$ are at the fourth decimal (Table 8), and Figure 5(a) shows the same flatness across six thresholds. This fixed-point property (C4) is exactly what distinguishes the probe from the iterative editors above — there is no stop time to calibrate and no rollout to drift — and it is why a single pass suffices.

Inference passes K	Mean Cov	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert/\mathrm{OPT}$
$K = 1$	0.9723	0.1968	1.2328
$K = 2$	0.9712	0.1966	1.2302
$K = 3$	0.9719	0.1966	1.2295
$K = 5$	0.9719	0.1966	1.2292

Table 8: Inference passes $K \in \{1, 2, 3, 5\}$ at fixed threshold $t = 0.65$, full `dev` split (1224 polygons). The variation across K is at the fourth decimal: iterating the `SetPredictor` returns essentially the same prediction. We report the K-sweep at $t \in \{0.5, 0.6, 0.65, 0.7, 0.75, 0.8\}$ rather than the headline range $\{0.20, 0.25, 0.30\}$ because the fixed-point property is structural — a consequence of the seed indicator already being one of the model's inputs — and we expect it to be most stringently tested near the decision boundary where small probability shifts could flip the most indicators; the headline thresholds admit more vertices and therefore involve smaller fractions of indicator flips between $K = 1$ and $K = 2$. Figure 5(a) shows the same K-invariance holds across all six thresholds in this range.

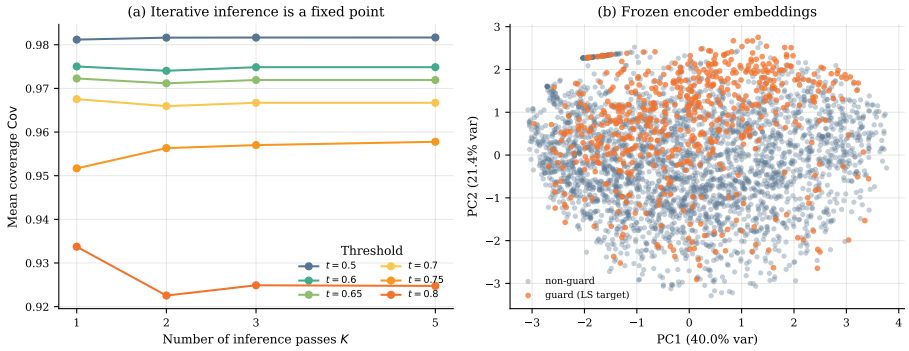

Figure 5: Two views of the probe's mechanism. *(a)* Iterative inference is a fixed point: mean coverage as a function of inference passes $K \in \{1, 2, 3, 5\}$ for six thresholds $t \in \{0.5, 0.6, 0.65, 0.7, 0.75, 0.8\}$, on the full `dev` split; the across-K variation is at the fourth decimal. *(b)* Two-dimensional projection of the frozen encoder's per-vertex embeddings, colored by guard-membership label (LS-derived target), pooled across held-out polygons. The separation visible here is corroborated quantitatively by a linear probe over the same 128-d embeddings (ROC-AUC 0.842 ± 0.024, Section 7); the 2D scatter is the visualization, the AUC is the measurement.

7 Discussion

The single most informative measurement in this paper is what happens when we read the encoder rather than the decoder. The `SetPredictor` probe sees only frozen encoder embeddings and the policy's own seed indicator — no visibility matrix, no CGAL output, no per-vertex area feature — yet it closes the in-distribution feasibility tail on the displayed seed (Table 3) and compresses the out-of-distribution feasibility tail by roughly an order of magnitude on average across four probe seeds (Section 6.3). Two readings are consistent with this. Either the encoder learned enough geometric structure that a small classifier over its output can recover feasibility on its own, or the decoder's EOS calibration was the bottleneck and the policy had already identified the right vertices but stopped too early. Both support the same conclusion about what the reinforcement method internalized: a representation that contains more of the relevant geometry than the policy's decoder expressed. Two further measurements quantify this. First, the coordinate-only ablation in Section 6.2 shows that masking the encoder embedding to zeros, with everything else held constant, leaves a probe that cannot match the full probe's coverage at matched cardinality on either split. Second, fitting a plain L_2-regularized logistic regression on the frozen per-vertex embeddings against the LS-derived guard-membership label achieves ROC-AUC $= 0.842 \pm 0.024$ (5-fold polygon-grouped cross-validation, 200 polygons, 12,268 vertices, mean $\pm$ std across folds; PR-AUC $= 0.580 \pm 0.048$ against a positive-class prior of 0.16). Figure 5(b) visualizes the same signal in a 2D projection.

Recovering feasibility has a price, and we report it rather than disguise it. The probe's guard sets grow to roughly two to three times the optimum as the threshold is lowered (Section 6.2), and the $|S|/\mathrm{OPT}$ box plots of Figure 3 show that the whole distribution widens, not just its mean. A deployment would pick one operating point on this curve; we leave it as a knob because the right point depends on how costly an extra guard is relative to an uncovered region.

We are not claiming a better AGP solver. Classical greedy reaches mean coverage 0.999 at $|S|/n = 0.152$ and $|S|/\mathrm{OPT} = 1.009$ on `dev_test` (Table 3); the probe at $t = 0.20$ reaches mean coverage ≥ 0.998 at $|S|/\mathrm{OPT} \approx 2.6$ across four seeds. The classical anchors win on cardinality at preserved coverage, and the policy and probe are not built to contest that. What we document is what is learnable by reinforcement when the model is denied geometric input at the moment of placing guards, and what part of that learning lives in the representation rather than the decoder. The contrast between the failed iterative editors (Section 6.4) and the single-shot probe is the cleanest expression of the point: removing the stop time and the rollout is exactly what lets the encoder's information surface.

8 Limitations

1. **Scope of the "no geometric knowledge" claim.** The geo-free constraint is on inference: the policy and the probe see no visibility object at test time. *Training* does see visibility: the PO/BT reward is computed from disc-vis coverage, and the probe's supervised targets are generated by local search using disc-vis as a coverage gate. "No explicit geometric knowledge" must be read as "no geometric oracle at the moment of placing guards."

2. **The probe is supervised, not reinforcement.** The `SetPredictor` is trained by BCE against local-search-derived labels. It is not itself a reinforcement learner. We position it as a representation probe of the RL-trained encoder, not as an RL contribution.

3. **Held-out partition size.** `dev_test` has 367 polygons. Wilson 95% CIs on the coverage-feasibility proportion are reported, but the absolute count is modest. The OOD evaluation on `test` (2107 polygons) provides a much larger second held-out evidence base.

4. **Policy selection and seeds.** The PO/BT policy is the best of several reinforcement-pretraining runs, selected by training-set performance. Because selection used only the `train` split, `dev_test` and `test` play no role in it and the held-out evaluations are not optimistically biased by the choice; selecting on training reward rather than a held-out criterion is a weaker rule that, if anything, risks favoring an over-fit run, so we make no claim the selected policy is optimal. We do not average over policy seeds; seed-variance is characterized separately for the probe over four seeds $\{1234, 11, 22, 33\}$ (Table 3, Section 6.3).

5. **LSTM, not Transformer, policy.** We trialed a Transformer encoder–decoder policy of roughly $6\times$ the parameters; it reached a comparable coverage–cost operating point (greedy coverage ≈ 0.97 at $|S|/\mathrm{OPT} \approx 1.15$ versus the LSTM pointer's ≈ 1.21 — sparser but no better on the trade-off), one training run failed to converge, and we found no improvement justifying the added capacity. We read this as overparametrization relative to the scalar RL signal rather than evidence against attention models in general — consistent with the fact that the probe, a small Transformer trained on dense per-vertex labels, does work; a careful attention-model study is future work.

6. **disc-vis approximation in training.** Both the PO/BT coverage reward and the `SetPredictor` training targets use disc-vis sampling at $M = 500$. Exact CGAL visibility is used only at evaluation.

7. **Extreme OOD is strong but seed-dependent.** The `large` split (285 polygons, n up to 2250) is now evaluated (Table 7, Section 6.3): the frozen-encoder probe lifts the worst-case coverage on every seed while the no-encoder ablation cannot, but the feasibility count is seed-variable ($253.5 \pm$

19.9 of 285, ranging from 0 to 55 below feasibility across the four seeds), so we report it as evidence of the encoder signal rather than a feasibility guarantee at this scale. Two caveats remain: the probe pays roughly double the local-search reference's guard count to reach this coverage, and the vertex-guard optimum is unavailable for 79 of these polygons (all with $n \geq 800$), so no approximation ratio is reported there.

8. **Training-curve reconstruction.** Figure 1 is reconstructed from four saved checkpoints (epochs 110, 114, 160, 200), not from an online metric log; per-epoch coverage was not preserved during the original PO/BT run. The trace shows only late-training dynamics, which is sufficient to confirm that the gradient direction does not collapse but does not characterize early-training behavior.

9 Conclusion

We asked what a neural policy internalizes about vertex-guard placement when trained from a coverage-aware reward over its own rollouts and denied any geometric oracle at inference, and we answered it by reading the encoder rather than the decoder: a single-shot probe over the frozen embeddings closes the in-distribution feasibility tail on the displayed seed and compresses the out-of-distribution tail by roughly an order of magnitude across four probe seeds, which we take as evidence that the reinforcement-trained representation already carries more of the geometry than the policy's decoder revealed. The result is a measurement in a deliberately constrained setting, not a competitor to classical solvers on guard count — the learner is not the best AGP heuristic, but it arrived at a usable representation of vertex-guard placement without ever being shown the geometry it had to reason about at inference. Two extensions follow naturally: a stronger encoder, such as a Transformer trained under the same PO/BT objective, may shorten the residual tail directly and leave less work for the probe; and a per-instance threshold predicted from the same embeddings would replace the global knob with an adaptive one.

References

[1] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.

[2] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

[3] Bahram Sadeghi Bigham, S. Dolatikalan, and Alireza Khastan. Minimum landmarks for robot localization in orthogonal environments. *Evol. Intell.*, 15(3):2235–2238, 2022.

[4] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3–4):324–345, 1952.

[5] Marcelo C. Couto, Pedro J. de Rezende, and Cid C. de Souza. Instances for the Art Gallery Problem. `http://www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery`, 2009.

[6] A. Elnagar and L. Lulu. An art gallery-based approach to autonomous robot motion planning in global environments. In *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 2079–2084, 2005.

[7] Héctor González-Baños. A randomized art-gallery algorithm for sensor placement. *Proceedings of the 17th Annual Symposium on Computational Geometry (SoCG)*, pages 232–240, 2001.

[8] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*, 2019.

[9] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. POMO: Policy optimization with multiple optima for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

[10] D. T. Lee and A. K. Lin. Computational complexity of art gallery problems. *IEEE Transactions on Information Theory*, 32(2):276–282, 1986.

[11] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[12] Joseph O'Rourke. *Art Gallery Theorems and Algorithms*. Oxford University Press, 1987.

[13] Yu Pan et al. Preference optimization for neural combinatorial optimization. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[14] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011.

[15] The CGAL Project. CGAL user and reference manual. `https://doc.cgal.org/5.6/Manual/packages.html`, 2024.

[16] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

[17] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.